Урок №25

Хранение данных в настройках приложения

О чём этот урок?

Представьте, что вы открыли приложение, ввели своё имя, выбрали тему оформления или настроили уведомления. Затем закрыли приложение и открыли его снова.

Было бы неудобно, если бы все настройки приходилось задавать заново при каждом запуске.

Сегодня познакомимся с **SharedPreferences** - механизмом Android для хранения небольших объёмов данных в виде пар "ключ - значение".

Научимся сохранять настройки, получать их при запуске приложения и изменять сохранённые значения.

Часть 1. Зачем нужно хранение данных?

1.1. Переменные и сохранённые данные

Вспомним обычные переменные Java:

```
String username = "Иван";
boolean darkMode = true;
```

В переменных хранятся значения, с которыми программа работает во время выполнения. Но если приложение закрыть, а затем запустить снова, оно не сможет автоматически восстановить эти значения из обычных переменных.

Чтобы данные сохранялись между запусками, их необходимо записать в постоянное хранилище.

Например, приложение может запоминать:

- Имя пользователя
- Выбранную тему оформления
- Размер текста
- Состояние переключателя уведомлений
- Другие небольшие настройки

Главная задача хранения данных - сохранить нужную информацию так, чтобы приложение могло получить её в дальнейшем.

1.2. Какие способы хранения существуют?

В Android есть несколько способов хранения данных. Они предназначены для разных задач.

Способ	Для чего используется
`SharedPreferences`	Небольшие настройки и отдельные значения
Файлы	Текстовые документы, изображения и другие данные
Локальная база данных	Структурированные записи, поиск, изменение и удаление данных
Хранилища для сериализованных объектов	Сохранение объектов в определённом формате

Сегодня изучаем только `SharedPreferences`. В дальнейшем рассмотрим другие способы хранения.

Часть 2. Что такое SharedPreferences?

2.1. Принцип работы

SharedPreferences - это механизм Android, который позволяет сохранять небольшие значения в виде пар "ключ - значение".

Представим, что мы разрабатываем приложение для заметок. Пользователь может выбрать своё имя и включить тёмную тему.

В настройках приложения могут храниться следующие данные:

Ключ	Значение
`username`	`Иван`
`dark_mode`	`true`
`notifications`	`false`

У каждой настройки есть ключ и значение.

- **Ключ** - название, по которому приложение находит нужную настройку.
- **Значение** - данные, которые необходимо сохранить.

Например, ключ `username` позволяет получить сохранённое имя пользователя.

Важно понимать, что ключ - это не само значение. По ключу `username` хранится строка `Иван`.

2.2. Как это выглядит в работе?

Допустим, пользователь включил тёмную тему.

1. Приложение сохраняет значение `true` по ключу `dark_mode`.
2. Пользователь закрывает приложение.
3. При следующем запуске приложение читает значение по ключу `dark_mode`.
4. Если сохранено `true`, приложение может включить тёмную тему.

Таким образом, приложение не просто запоминает выбор во время работы, а сохраняет его для последующих запусков.

Важно: SharedPreferences хранит данные, но само по себе не изменяет интерфейс. Приложение должно прочитать настройку и самостоятельно применить её.

Часть 3. Работа с SharedPreferences

3.1. Получение хранилища

Чтобы работать с настройками, сначала необходимо получить объект SharedPreferences.

```
SharedPreferences preferences =
        getSharedPreferences("settings", MODE_PRIVATE);
```

Разберём этот код.

- `getSharedPreferences()` - метод для получения хранилища настроек.
- `"settings"` - имя хранилища.
- `MODE_PRIVATE` - режим, при котором хранилище предназначено для доступа из текущего приложения.

Если хранилища с таким именем ещё нет, оно будет создано при первой записи данных.

3.2. Сохранение данных

Для изменения настроек используется объект `Editor`.

Пример сохранения имени пользователя:

```
SharedPreferences.Editor editor =
        preferences.edit();

editor.putString("username", "Иван");

editor.apply();
```

Рассмотрим последовательность действий:

1. `edit()` - получаем объект для изменения настроек.
2. `putString()` - указываем ключ и значение, которое необходимо сохранить.
3. `apply()` - применяем изменения и сохраняем их асинхронно.

После выполнения этого кода приложение сможет получить строку `Иван` по ключу `username`.

Обратите внимание: вызов `putString()` подготавливает изменение, но для его применения необходимо вызвать `apply()` или `commit()`.

Для обычного сохранения настроек чаще используют `apply()`, поскольку он не ожидает завершения записи на основном потоке.

3.3. Какие типы данных можно сохранять?

SharedPreferences поддерживает несколько простых типов данных.

Метод	Тип данных	Пример
`putString()`	`String`	`"Иван"`
`putInt()`	`int`	`5`
`putBoolean()`	`boolean`	`true`
`putFloat()`	`float`	`18.5f`
`putLong()`	`long`	`100000L`

Например, можно сохранить имя, количество запусков приложения и состояние переключателя:

```
preferences.edit()
        .putString("username", "Иван")
        .putInt("launch_count", 5)
        .putBoolean("dark_mode", true)
        .apply();
```

В этом примере все три значения сохраняются за один вызов `apply()`.

3.4. Получение сохранённых данных

Сохранить значение недостаточно. Приложению необходимо уметь получать его.

Для чтения используется метод, соответствующий типу данных.

```
String username =
        preferences.getString("username", "Гость");

int launchCount =
        preferences.getInt("launch_count", 0);

boolean darkMode =
        preferences.getBoolean("dark_mode", false);
```

В методах чтения указываются два аргумента:

1. Ключ, по которому необходимо получить значение.
2. Значение по умолчанию, если соответствующий ключ отсутствует.

Например:

```
String username =
        preferences.getString("username", "Гость");
```

Если по ключу `username` сохранено имя `Иван`, переменная получит значение `Иван`.

Если такого ключа нет, переменная получит значение `Гость`.

Значение по умолчанию не записывается в хранилище автоматически. Оно используется только тогда, когда сохранённое значение отсутствует.

3.5. Изменение сохранённых данных

Чтобы изменить настройку, можно сохранить новое значение по тому же ключу.

```
preferences.edit()
        .putString("username", "Пётр")
        .apply();
```

Теперь по ключу `username` будет храниться строка `Пётр`, а не `Иван`.

Создавать новый ключ для каждого изменения не требуется.

3.6. Удаление данных

Иногда пользователю необходимо сбросить настройку.

Для удаления одного значения используется `remove()`:

```
preferences.edit()
        .remove("username")
        .apply();
```

После удаления ключ `username` больше не будет содержать сохранённого значения. При чтении можно будет получить значение по умолчанию.

Для удаления всех значений из выбранного хранилища используется `clear()`:

```
preferences.edit()
        .clear()
        .apply();
```

Будьте внимательны: `clear()` удаляет все настройки из данного хранилища. Если в нём находятся другие важные параметры, они также будут удалены.

Часть 4. Ограничения SharedPreferences

4.1. Для каких задач подходит SharedPreferences?

SharedPreferences удобно использовать для хранения небольших настроек:

- Имя пользователя.
- Тема оформления.
- Состояние уведомлений.
- Предпочитаемый размер текста.
- Простые параметры поведения приложения.

4.2. Когда лучше использовать другой способ?

SharedPreferences не предназначен для хранения больших коллекций объектов и сложных структур данных.

Например, если приложение содержит сотни заметок, для которых нужны поиск, редактирование, удаление и сортировка, лучше использовать локальную базу данных.

В последующих занятиях мы познакомимся с другими способами хранения данных и научимся выбирать подходящий вариант для конкретной задачи.

4.3. Что происходит при удалении приложения?

Данные SharedPreferences относятся к конкретному приложению. Обычно они сохраняются между обычными запусками, но могут быть удалены при очистке данных приложения или его удалении.

Также SharedPreferences не следует считать защищённым хранилищем секретов. Пароли и другие чувствительные данные нельзя хранить там в обычном виде.

Итог

- **SharedPreferences** - механизм хранения небольших значений в формате "ключ - значение".
- `getSharedPreferences()` - получение хранилища.
- `putString()`, `putInt()`, `putBoolean()` - сохранение значений.
- `getString()`, `getInt()`, `getBoolean()` - чтение значений.
- `apply()` - применение изменений.
- `remove()` - удаление одного значения.
- `clear()` - очистка выбранного хранилища.